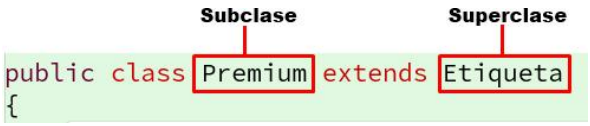
**Nombre del Alumno:** Morales, Francisco
**DNI:** 43.533.393
**Nombre del Alumno:** Romero, Enzo
**DNI:** 41.948.148.
**Nombre del Profesor/a:** Yanina Medina – Andrea Lezcano Airaldi
**Grupo de Laboratorio:** 2
**TP:** 3
**Fecha de entrega:** 06/09/2026

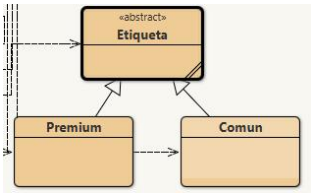
Para este informe usaremos como referencia el ejercicio 11 de la guía.

## Herencia

Herencia se refiere al mecanismo de compartir atributos y métodos utilizando la relación de generalización.
En el ejercicio 11 tenemos la superclase **Etiqueta** y las dos subclases **Premium** y **Común**.
Mediante **extends**, ambas heredan atributos y métodos de **Etiqueta**, como **costo** y **getCosto()**.
Ademas, cada subclase agrega o define su propio comportamiento, por ejemplo, implementando.




Ademas, en el UML vamos a ver a estas subclases representadas con una flecha de punta cerrada apuntando a la superclase, por lo que es fácil identificarlas.




## Polimorfismo

Se refiere a la posibilidad de que objetos de clases diferentes respondan a mensajes con el mismo nombre, cada uno de ellos con su propio comportamiento.

Como ejemplo se observa el polimorfismo en el método **precio(int q)** de la clase **Etiqueta**, ya que tanto los objetos de la clase **Premium** como los de la clase **Comun** responden al mismo mensaje, pero cada uno lo hace con su propia complementación.

**precio(int q) en Premium**

```
public double precio(int q){
    double precio;
    double adicional;
    precio = getCosto() * q;
    adicional = precio * this.plus();
    return precio + adicional;
}
```

**precio(int q) en Comun**

```
public double precio(int q){
    double precio;
    precio = this.getCosto() * q + this.getPlus();
    precio = precio - this.descuento(q);
    return precio;
}
```

## Clases Abstractas

En ocasiones es necesario definir clases para las cuales no tiene sentido crear instancias. Estas clases se construyen para generalizar estructura y comportamiento común de varias clases descendientes.

En este caso**, Etiqueta** es una clase abstracta porque representa una generalización de las etiquetas comunes y premium.

```
public abstract class Etiqueta
```

Usamos Abstract para declararlas.

## Metodos abstractos

Un método abstracto es aquel que no tiene implementación, y que por lo tanto nunca será ejecutado.

En la clase **Etiqueta** tenemos algunos de estos métodos como **tipo()**. Donde usamos **Abstract** para declararlo.

```
protected abstract String tipo();
```

## Pseudovariable super

Permite acceder a los miembros de la clase padre, como sus constructores o métodos.

```
public Premium(int p_colores, double p_costo)
{
    super(p_costo);
    this.setColores(p_colores);
}
```

En la clase **premium** podemos ver que **super(p_costo)** invoca al constructor de la clase padre.

Su diferencia con **this** es que **this** hace referencia al objeto actual, mientras que **super** permite acceder a la implementación heredada de la clase padre.